Fundamentals of API Development

Breakout Lab 1.2: Writing an OpenAPI Specification

REFERENCE ONLY

Day 2

KavinSchool

Lab Overview

Goal: Write a complete OpenAPI 3.1 specification for the quickstart API's two real endpoints, `health` and `greetings`, with reusable schemas and proper `$ref` usage.

- **Format:** Individual or pair work
- **Tools:** Swagger Editor (or IDE OpenAPI plugin), Postman, the quickstart API running on port 8080

> **NOTE**
>
> You just watched `DemoOpenApiSpecValidator` validate a hand-written spec and `DemoSchemaReuse` show how `$ref` prevents drift. This lab asks you to write the spec those demos validated and reused.

What's in the Companion Repo

Open `day2/lab1_2_writing_an_openapi_specification/` in the companion repo.

Path	What it is
`start/openapi-quickstart.yaml`	The spec you complete. `/health` is done as a worked example; `/greetings` and its schema are marked `TODO`.
`solution/openapi-quickstart.yaml`	A completed spec for both endpoints. Open after you have tried.

SETUP

Start the API: `mvn spring-boot:run` in `api-dev-setup/quickstart-project`. Validate as you go by pasting into the Swagger Editor.

Step 1: Observe the Real Behavior First

Send Both Requests

Send both requests in Postman and note the exact response shape. Your spec must describe what the API actually returns, not what you assume.

```
curl http://localhost:8080/api/v1/health
curl http://localhost:8080/api/v1/greetings/YourName
```

EVIDENCE FIRST

The same discipline Lab 1.1 asked for: every field in your spec must match a field you saw in a real response.

Step 2: Read the Worked Example

The Health Endpoint Is Done

`start/openapi-quickstart.yaml` already has `/api/v1/health` fully specified:

- A `summary` and `description` for the operation
- A `200` response with content referencing a `HealthStatus` schema
- The `HealthStatus` schema defined once under `components/schemas`

This is the pattern you are about to repeat for the greetings endpoint. Read it before writing anything.

Step 3: Complete the Greetings Endpoint

Fill In Every TODO (1/2)

For `/api/v1/greetings/{name}`:

1. **`summary` and `description`** for the operation, tell a consumer what it does.
2. **The `name` path parameter**, mark it `required`, give it a `string` schema and a one-line `description`.
3. **A `200` response** whose `content` references a `Greeting` schema via `$ref`.
4. **The `Greeting` schema** under `components/schemas`, with a `message` field matching the response you observed in Step 1.

Fill In Every TODO (2/2)

CHECK YOUR WORK

Paste your file into the Swagger Editor. Fix anything it flags. Then compare against `solution/openapi-quickstart.yaml`, not to copy it, but to see whether your descriptions would actually help a consumer who has never seen this API.

Stretch Goals

If you finish early:

1. Add a `404` response to the greetings path. The real API currently returns a generic Spring error body for unmapped paths, describe what you observe, not what you wish it returned.
2. Add a second example value to the `Greeting` schema. Pick a name different from the one in the worked example.

Lab Completion Checklist

Checklist (1/2)

- [ ] Both `/health` and `/greetings/{name}` requests sent, response shape recorded
- [ ] Worked example (`/health`) read and understood
- [ ] All four TODOs filled in for the greetings endpoint
- [ ] Spec validated in the Swagger Editor with zero errors
- [ ] `Greeting` schema uses `$ref` , not duplicated inline

Checklist (2/2)

DONE?

Save your `openapi-quickstart.yaml`. You will trade it with a partner in Lab 1.3, peer review it, then compare it against the version springdoc-openapi generates.

Lab 1.2 Complete

Bring your spec to Lab 1.3 for peer review

KavinSchool